

Exp: 06: Hamming Code

7/8/26

Aim:-

To write a program to implement error detection and correction using Hamming Code concept.

~~Write a sender program using~~

Sender.py:-

```
def find_r(k):
```

```
    r=0
```

```
    while  $2^{*r} < k + r + 1$ :
```

```
        r += 1
```

```
    return r
```

```
text = input("Enter text: ")
```

```
binary = ""
```

```
for ch in text: ("n/" + (i) % 10) % 10
```

```
    binary += format(ord(ch), "08b")
```

```
print("Binary:", binary)
```

```
k = len(binary)
```

```
r = find_r(k)
```

```
print("Data bits:", k)
```

```
print("Redundancy bits:", r)
```

```
n = k + r
```

```
code = [0] * (n + 1)
```

```
j = 0
```

```
for i in range(1, n + 1):
```

```
    if  $i \& (i - 1) == 0$ :
```

```
        code[i] = int(binary[j])
```

```
        j += 1
```

```
p = 1
```

```
while p <= n:
```

```
    parity = 0
```

```
    for i in range(p, n+1):
```

```
        if i & p:
```

```
            parity ^= code[i]
```

```
    code[p] = parity
```

```
    p = p * 2
```

```
hamming-code = ""
```

```
for i in range(1, n+1):
```

```
    hamming-code += str(code[i])
```

```
print("Hamming code: ", hamming-code)
```

```
with open("channel.txt", "w") as f:
```

```
    f.write(str(k) + "\n")
```

```
    f.write(hamming-code)
```

```
print("Data saved in channel.txt")
```

Receiver.py

```
def binary_to_text(binary):
```

```
    text = ""
```

```
    for i in range(0, len(binary), 8):
```

```
        byte = binary[i:i+8]
```

```
        if len(byte) == 8:
```

```
            text += chr(int(byte, 2))
```

```
    return text
```

```
with open("channel.txt", "r") as f:
```

```
    k = int(f.readline())
```

```
    data = f.readline().strip()
```

```
print("Received Hamming: (code: ", data)
```

```
n = len(data)
```

```
code = [0] + [int(x) for x in data]
```

```
corrupted_binary = ""
```

```
for i in range(1, n+1):
```

```
    if i & (i-1) != 0:
```

```
        corrupted_binary += str(code[i])
```

```
corrupted_binary = corrupted_binary[:K]
```

```
corrupted_text = binary_to_text(corrupted_binary)
```

```
print("Corrupted Text: ", corrupted_text)
```

```
error = 0
```

```
p = 1
```

```
while p <= n:
```

```
    parity = 0
```

```
    for i in range(p, n+1):
```

```
        if i & p:
```

```
            parity ^= code[i]
```

```
    if parity:
```

```
        error += p
```

```
    p = p * 2
```

```
if error == 0:
```

```
    print("No error detected.")
```

```
else:
```

```
    print("Error detected at position: ", error)
```

```
    code[error] = 1 - code[error]
```

```
    print("Error corrected.")
```

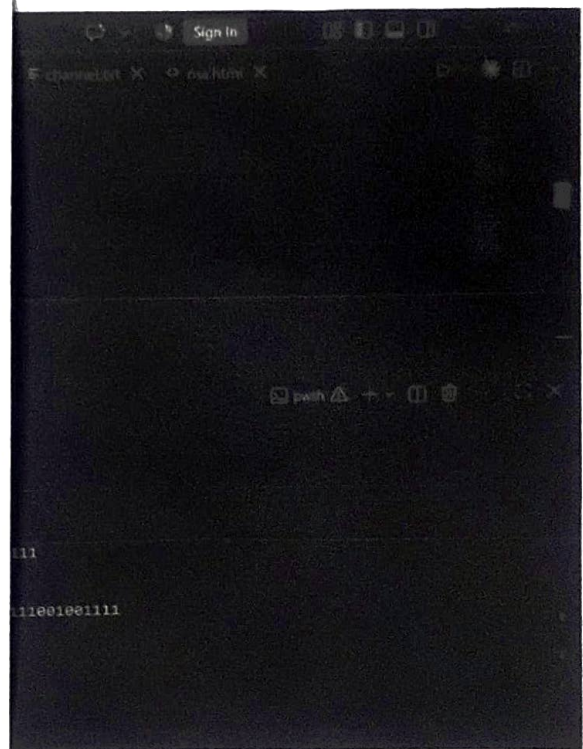
```
corrected_library = ""
```



```

for i in range(1, n+1):
    if i & (i-1) != 0:
        corrected_binary += str(code[i])
    corrected_binary = corrected_binary[:K]
corrected_text = binary_to_text(corrected_binary)
print("Corrected Text:", corrected_text)

```



Result:-

Thus, error detection and correction is done using hamming code.